

Ce sujet aborde différents problèmes autour des automates et des expressions régulières. On explore dans une première partie des propriétés sur le miroir d'un langage, sur les palindromes et sur les automates correspondants. On implémente ensuite l'algorithme de déterminisation d'un automate, afin de construire de manière effective l'automate de Brzowski qui a la propriété d'être minimal. Dans une deuxième partie, on travaille sur la syntaxe des expressions régulières, puis sur une construction de l'expression régulière associée à un automate donné, par un algorithme diviser pour régner, dû à Conway, en exploitant une représentation matricielle d'un automate et la construction de l'étoile d'une matrice d'expressions régulières. Enfin, dans une troisième partie, on introduit les dérivées d'Antimirov, qui permettent d'obtenir un automate fini non déterministe avec peu d'états qui reconnaît le langage spécifié par une expression régulière. Les trois parties sont indépendantes, de difficulté progressive.

Langages et mots

On appelle alphabet tout ensemble fini de lettres. On note généralement l'alphabet Σ . On note Σ^* l'ensemble de tous les mots formés sur Σ . La longueur (ou la taille) d'un mot $w \in \Sigma^*$ est son nombre de lettres et se note $|w|$. Le mot vide, noté ε , est le seul mot de longueur nulle. Si $|w| = n$, on note $w = a_0 a_1 \dots a_{n-1}$, où les a_i sont des lettres de Σ . Un langage sur l'alphabet Σ est un ensemble $L \subseteq \Sigma^*$. L'étoile de Kleene d'un langage L , notée L^* , est le plus petit langage qui inclut L , qui contient ε et qui est stable par concaténation. La concaténation de deux langages L et L' est notée $L \cdot L'$, souvent abrégée en LL' lorsqu'il n'y a pas d'ambiguïté.

Automates finis

Un automate fini non déterministe sur Σ est un quadruplet $A = (Q, I, F, T)$, où Q est un ensemble fini d'états, $I \subseteq Q$ l'ensemble des états initiaux, $F \subseteq Q$ l'ensemble des états finaux et $T \subseteq Q \times \Sigma \times Q$ l'ensemble des transitions. Si $(q, a, q') \in T$, on note cette transition $q \xrightarrow{a} q'$. Dans la figure 1, une flèche entrante désigne un état initial et un double cercle un état final.

Un mot $w = a_0 \dots a_{n-1}$ est reconnu par A s'il existe une succession de transitions

$$q_0 \xrightarrow{a_0} q_1 \xrightarrow{a_1} \dots q_{n-1} \xrightarrow{a_{n-1}} q_n, \quad q_0 \in I, \quad q_n \in F.$$

On dit que w étiquette un chemin de q_0 à q_n . Le langage de l'automate, noté L_A , est l'ensemble des mots reconnus par A . Un langage est reconnaissable s'il est le langage d'un automate fini.

Un automate déterministe est un quadruplet $A = (Q, \{q_0\}, F, \delta)$ avec un unique état initial et une fonction de transition δ définie sur une partie de $Q \times \Sigma$, à valeurs dans Q . Pour chaque (q, a) , il existe au plus une transition (q, a, q') ; si elle existe, $q' = \delta(q, a)$. L'automate est complet si δ est définie sur $Q \times \Sigma$. On définit alors sa fonction de transition étendue par

$$\delta^*(q, \varepsilon) = q, \quad \delta^*(q, wa) = \delta(\delta^*(q, w), a) \quad (q \in Q, w \in \Sigma^*, a \in \Sigma).$$

Les automates sont représentés par le type OCaml suivant ; l'ensemble d'états est toujours l'intervalle d'entiers $\llbracket 0, n-1 \rrbracket$:

```
type automate = {
  nb : int;
  init : int list;
  final : int list;
  trans : (int * char * int) list
}
```

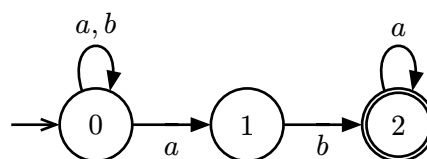


Figure 1 — L'automate $\mathcal{A}_1$.

Par exemple, l'automate de la figure 1 est codé par :

```
let a1 = {
  nb = 3; init = [0]; final = [2];
```

```

trans = [(0, 'a', 0); (0, 'a', 1); (0, 'b', 0);
         (1, 'b', 2); (2, 'a', 2)]
}

```

On accède au nombre d'états par `al.nb`, aux états initiaux par `al.init`, aux états finaux par `al.final` et aux transitions par `al.trans`.

Expressions régulières

Sur un alphabet Σ , $\emptyset$, ε et toute lettre $a \in \Sigma$ sont des expressions régulières. Si E, F sont des expressions régulières, alors $(E \mid F)$, $(E \cdot F)$ et E^* en sont aussi. L'application $\mathcal{L}$ associe à une expression le langage qu'elle représente :

$$\begin{aligned} \mathcal{L}(\emptyset) &= \emptyset, & \mathcal{L}(\varepsilon) &= \{\varepsilon\}, & \mathcal{L}(a) &= \{a\}, \\ \mathcal{L}(E \mid F) &= \mathcal{L}(E) \cup \mathcal{L}(F), & \mathcal{L}(E \cdot F) &= \mathcal{L}(E)\mathcal{L}(F), & \mathcal{L}(E^*) &= \mathcal{L}(E)^*. \end{aligned}$$

Programmation

Le seul langage de programmation autorisé est OCaml. Toutes les fonctions des modules `Array` et `List`, ainsi que les fonctions de la bibliothèque standard (comme `max` et `incr`) et les opérateurs comme `/` ou `mod` peuvent être utilisés. Les objets mathématiques notés A, m, i, n, ℓ sont représentés en OCaml par `a`, `m`, `i`, `n`, `l`. Les complexités demandées sont temporelles dans le pire des cas et sont exprimées sous la forme $O(f(n, m))$, où f est une fonction usuelle simple des tailles des objets en entrée.

I – Mots et automates

I.A – Miroir d'un mot et automate transposé

Pour tout mot $w = a_0 a_1 \dots a_{n-1}$ de longueur $n \in \mathbb{N}^*$, son miroir est $\tilde{w} = a_{n-1} \dots a_1 a_0$. Le mot vide ε est son propre miroir. Pour tout langage $L \subseteq \Sigma^*$, on pose $\tilde{L} = \{\tilde{w} \mid w \in L\}$.

1. Décrire le langage L_1 de l'automate $\mathcal{A}_1$ de la figure 1 et décrire son langage miroir $\tilde{L}_1$.
2. Dessiner un automate $\tilde{\mathcal{A}}_1$ reconnaissant $\tilde{L}_1$.

Soit $A = (Q, I, F, T)$ un automate non déterministe et $L = L_A$ son langage.

3. Donner, en justifiant, la construction de l'automate miroir $\tilde{A} = (Q, I', F', T')$ reconnaissant $\tilde{L}$.
4. Écrire une fonction `transpose` de signature `automate -> automate` qui renvoie un automate reconnaissant le miroir du langage de l'automate d'entrée.
5. Quelle est la complexité de cette fonction ?

I.B – Palindromes et régularité

Soit $w \in \Sigma^*$. Le mot w est un palindrome si $\tilde{w} = w$.

6. Écrire une fonction `palindrome` de signature `string -> bool` qui teste, en temps linéaire, si un mot est un palindrome.

On rappelle que pour $0 \leq i < \text{String.length } s$, le i -ième caractère de la chaîne s est obtenu par `s.[i]`. Pour un alphabet Σ , on note $\text{Pal}(\Sigma)$ l'ensemble des palindromes de Σ^* .

7. Montrer que si Σ est un alphabet à une lettre, alors $\text{Pal}(\Sigma)$ est régulier.
8. Montrer que si Σ contient au moins deux lettres, alors $\text{Pal}(\Sigma)$ n'est pas régulier. On pourra utiliser un automate et un mot de $\text{Pal}(\Sigma) \cap a^* b a^*$.

Soit $L \subseteq \Sigma^*$ un langage reconnu par $A = (Q, I, F, T)$. Pour $(q, q') \in Q^2$, on note $L_{q, q'}$ le langage des mots étiquetant un chemin de q à q' dans A .

9. Montrer que $L_{q, q'}$ est reconnaissable et exprimer L_A en fonction des langages $L_{q, q'}$.
10. Montrer que $\text{Pal}(\Sigma) \cap (\Sigma^2)^* = \{u\tilde{u} \mid u \in \Sigma^*\}$.

Soit L un langage régulier reconnu par $A = (Q, I, F, T)$. On définit $D(L) = \{w\tilde{w} \mid w \in L\}$ et $R(L) = \{w \in \Sigma^* \mid w\tilde{w} \in L\}$.

11. Décrire simplement $D(a^*b)$ et $R(a^*b^*a^*)$.
12. Les langages $D(L)$ et $R(L)$ sont-ils reconnaissables ? On pourra faire intervenir les langages $L_{q,q'}$ définis ci-dessus.

I.C – Déterminisation

Pour tout automate non déterministe $A = (Q, I, F, T)$, on définit le déterminisé accessible $A_{\text{det}} = (Y, \{I\}, F', \delta)$, où $Y \subseteq \mathcal{P}(Q)$ est l'ensemble des états accessibles depuis l'état initial I dans l'automate des parties. Il reconnaît le même langage que A .

13. Écrire un automate $\mathcal{A}_2$ non déterministe à quatre états reconnaissant $L_2 = (b \mid ab)^*ba$, avec un unique état initial et un unique état final.
14. Déterminiser l'automate miroir $\tilde{\mathcal{A}}_2$ pour obtenir $\mathcal{A}_3 = (\tilde{\mathcal{A}}_2)_{\text{det}}$. Renommer ses états $e_0, e_1, \dots$.
15. Déterminiser l'automate miroir $\tilde{\mathcal{A}}_3$ pour obtenir $\mathcal{A}_4 = (\tilde{\mathcal{A}}_3)_{\text{det}}$. Renommer ses états $q_0, q_1, \dots$.
16. Quel doit être le langage reconnu par $\mathcal{A}_4$?

On souhaite implémenter cette construction. Une représentation naïve des parties de Q utilise des listes d'états. Lors des réunions, la concaténation de listes crée des doublons qu'il faut supprimer.

17. Écrire `supprimer` : `'a list -> 'a list` qui supprime toutes les occurrences multiples des éléments d'une liste.
18. Donner la complexité en fonction de la taille de la liste.

On code désormais les ensembles d'états par des entiers. Pour $Q = \llbracket 0, n-1 \rrbracket$, avec $n \leq 20$, toute partie est représentée par un entier entre 0 et $2^n - 1$. On pose $\text{numero}(X) = \sum_{i \in X} 2^i$. Le tableau `pow` contient 2^k pour $0 \leq k \leq 20$.

```
let pow = Array.make 21 1
let () =
  for i = 1 to 20 do
    pow.(i) <- pow.(i - 1) * 2
  done
```

Soient $q \in \llbracket 0, n-1 \rrbracket$ et $k = \text{numero}(X) \in \llbracket 0, 2^n - 1 \rrbracket$.

19. Écrire `est_dans` : `int -> int -> bool` qui teste, à l'aide d'opérations arithmétiques, si $q \in X$ en $O(1)$ opérations.

Soit ℓ une liste d'états pouvant contenir des doublons, représentant X .

20. Écrire `numero` : `int list -> int` qui calcule le numéro de X . Par exemple, `[1; 5; 2; 5; 2; 5; 2; 2; 1; 2; 1]` représente $\{1, 2, 5\}$, de numéro $38 = 2^1 + 2^2 + 2^5$.

Soit ℓ une liste d'états et X un ensemble représenté par k .

21. Écrire `intersecte` : `int list -> int -> bool` qui vérifie si un élément de ℓ appartient à X .

On suppose désormais $\Sigma = \{a, b\}$. Pour $X \subseteq Q$ et $c \in \Sigma$, la transition du déterminisé est

$$\delta(X, c) = \cup_{q \in X} \{q' \in Q \mid (q, c, q') \in T\}.$$

En parcourant T , on calcule simultanément $\delta(X, a)$ et $\delta(X, b)$.

22. Écrire `etat_suivant` : `int -> (int * char * int) list -> int * int` qui, à partir de $k = \text{numero}(X)$ et de T , renvoie $(k_a, k_b) = (\text{numero}(\delta(X, a)), \text{numero}(\delta(X, b)))$.

Pour obtenir un ensemble d'états $Y = \llbracket 0, N - 1 \rrbracket$, on renumérote les parties accessibles avec une liste de couples $(k, v) : k$ code une partie, v est son nouveau numéro. Par exemple, $(6, 2)$ renumérote la partie $\{1, 2\}$, de code 6, en l'état 2.

23. Écrire `cherche` : `int -> (int * int) list -> int` qui renvoie le nouveau numéro associé à k , ou -1 si k est absent.
24. Écrire `determinise` : `automate -> automate` qui calcule le déterminisé accessible. Expliquer brièvement la démarche.
25. Quelle est la complexité de `determinise` en fonction du nombre n d'états de A et du nombre N d'états de A_{det} ?

I.D – Algorithme de Brzozowski

L'algorithme de Brzozowski donne un automate déterministe complet ayant un nombre minimal d'états parmi les automates déterministes complets reconnaissant le même langage. On se donne $A = (Q, I, \{f\}, T)$ reconnaissant L , dont le miroir $\tilde{A}$ est déterministe et accessible. On note $A_{\text{det}} = (Y, \{I\}, F', \delta)$ son déterminisé accessible. Pour $u \in \Sigma^*$, on pose $u^{-1}L = \{w \in \Sigma^* \mid uw \in L\}$.

26. Soient $q \in Q$ et $u \in \Sigma^*$. Montrer que si $q \in \delta^*(I, u)$, alors il existe $w \in \Sigma^*$ tel que $uw \in L$.
27. Montrer la propriété $(*)$: si $u^{-1}L = v^{-1}L$, alors $\delta^*(I, u) = \delta^*(I, v)$.
28. Pour un automate quelconque A reconnaissant L , on pose $B = (\tilde{A})_{\text{det}}$. En déduire que $(\tilde{B})_{\text{det}}$ reconnaît L et vérifie $(*)$.
29. Écrire `minimal` : `automate -> automate` appliquant cette construction. On fera abstraction de la taille des automates générés, possiblement problématique.

II – Expression régulière associée à un automate

On introduit un algorithme dû à Conway, calculant une expression régulière associée au langage d'un automate à l'aide de matrices d'expressions régulières.

II.A – Simplification d'expressions régulières équivalentes

On se donne le type OCaml des expressions régulières :

```
type exprat =
  | Vide
  | Epsilon
  | Lettre of char
  | Union of exprat * exprat
  | Concat of exprat * exprat
  | Etoile of exprat
```

II.A.1 – Parcours d'une expression

30. Écrire `lettre` : `exprat -> int` qui renvoie le nombre de lettres présentes dans une expression régulière. Par exemple, pour $E = (a^*b) \mid abba(a \mid \varepsilon)^* \mid \emptyset$, la fonction doit renvoyer 7.
31. Écrire `est_vide` : `exprat -> bool` qui teste si le langage représenté est vide.

II.A.2 – Règles de simplification

On travaille sur la syntaxe des expressions et on utilise les équivalences

$$\emptyset \mid E \equiv E \mid \emptyset \equiv E, \quad E \cdot \varepsilon \equiv \varepsilon \cdot E \equiv E,$$

$$E \cdot \emptyset \equiv \emptyset \cdot E \equiv \emptyset, \quad \emptyset^* \equiv \varepsilon, \quad \varepsilon^* \equiv \varepsilon, \quad (E^*)^* \equiv E^*.$$

La notation $E \equiv E'$ signifie $\mathcal{L}(E) = \mathcal{L}(E')$. La fonction suivante simplifie à la racine une union :

```

let su = function
| Union (Vide, e) | Union (e, Vide) -> e
| e -> e

```

On suppose aussi codée `sc : exprat -> exprat`, qui simplifie à la racine une concaténation en utilisant les règles données.

32. Écrire `se : exprat -> exprat` qui simplifie à la racine une expression de type `Etoile` selon ces règles.

Considérons $E_n = a \mid (b \cdot (b \cdot (\dots (b \cdot \emptyset) \dots)))$, où n lettres b concaténées se succèdent.

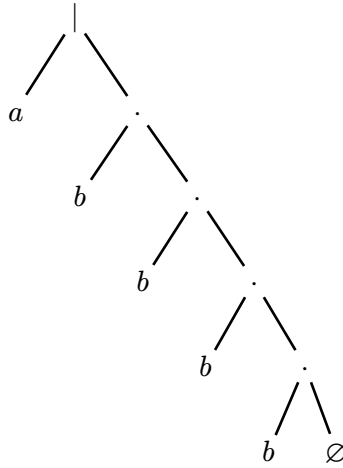


Figure 2 — Arbre syntaxique de E_4 .

33. Combien d'applications des règles sont nécessaires pour obtenir l'expression a à partir de E_n ?

34. Écrire `simplifie : exprat -> exprat` qui simplifie une expression selon ces règles.

II.B – Matrices d'expressions régulières

On considère des matrices d'expressions régulières :

```

type mat = exprat array array

```

La matrice nulle de taille n a tous ses coefficients égaux à $\emptyset$. La matrice identité a ε sur sa diagonale et $\emptyset$ ailleurs. Pour A, B de taille $n \times m$, on définit $[A \mid B]_{i,j} = A_{i,j} \mid B_{i,j}$, où le signe $\mid$ désigne l'union, pour $0 \leq i < n$ et $0 \leq j < m$. Pour A de taille $n \times p$ et B de taille $p \times q$, le produit de taille $n \times q$ est défini comme le produit usuel, en remplaçant la somme par l'union et le produit par la concaténation.

Les dimensions de cette sous-partie sont strictement positives. Pour les complexités, on compte la construction d'un nœud d'expression en temps constant, avec partage des sous-expressions immuables. On ne développe ni n'imprime les expressions pendant les calculs.

II.B.1 – Somme et produit

35. Écrire `somme : mat -> mat -> mat` qui effectue la somme de matrices de même taille $n \times p$. Quelle est sa complexité ?

36. Écrire `produit : mat -> mat -> mat` qui effectue le produit de deux matrices de tailles compatibles $n \times p$ et $p \times q$. On ne vérifiera pas cette compatibilité. Quelle est sa complexité ?

On cherche à définir l'étoile de Kleene d'une matrice carrée.

II.B.2 – Étoile d'une matrice de taille deux

Soit $M = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$, où a, b, c, d sont quatre lettres. On associe à M le graphe étiqueté $G = (S, A)$ de la figure 3, avec $S = \{0, 1\}$.

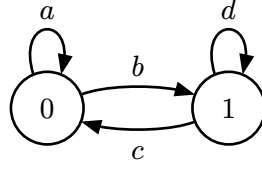


Figure 3 — Graphe associé à M .

On note $L_{i,j}$ le langage de l'automate $\mathcal{A}_{i,j} = (\{0, 1\}, \{i\}, \{j\}, T)$, où $T = \{(i, M_{i,j}, j) \mid (i, j) \in \{0, 1\}^2\}$.

37. Donner une expression régulière sur $\{a, b, c, d\}$ pour chacun des langages $L_{i,j}$.

II.B.3 – Étoile d'une matrice de taille quelconque

On définit récursivement l'étoile de M : si $M = (e)$ est de taille 1, alors $M^* = (e^*)$. Sinon, on découpe en blocs

$$M = \begin{pmatrix} A & B \\ C & D \end{pmatrix}, \quad M^* = \begin{pmatrix} A' & B' \\ C' & D' \end{pmatrix},$$

où A, D sont carrées de tailles au moins 1, et

$$\begin{aligned} A' &= (A \mid BD^*C)^*, & B' &= A^*B(D \mid CA^*B)^*, \\ C' &= D^*C(A \mid BD^*C)^*, & D' &= (D \mid CA^*B)^*. \end{aligned}$$

Les fonctions suivantes sont supposées codées :

- `decouper : mat -> int -> int -> mat * mat * mat * mat` : découper m en n_1 n_2 renvoie les blocs A, B, C, D d'une matrice carrée M de taille $n_1 + n_2$, avec A carrée de taille n_1 et D carrée de taille n_2 ;
- `recoller : mat -> mat -> mat -> mat -> mat` : reconstruit $M = \begin{pmatrix} A & B \\ C & D \end{pmatrix}$ à partir de quatre blocs de tailles compatibles.

Pour une matrice de taille $n \geq 2$, considérons d'abord

$$M = \begin{pmatrix} a & B \\ C & D \end{pmatrix},$$

où a est une expression régulière (un bloc de taille 1) et D est carrée de taille $n - 1$. On a alors

$$\begin{aligned} A' &= (a \mid BD^*C)^*, & B' &= a^*B(D \mid Ca^*B)^*, \\ C' &= D^*C(a \mid BD^*C)^*, & D' &= (D \mid Ca^*B)^*. \end{aligned}$$

38. Évaluer les complexités des sommes et produits. En déduire que le coût $C(n)$ du calcul de l'étoile vérifie $C(n) = 2C(n-1) + O(n^2)$. En déduire la complexité de cet algorithme.

Supposons maintenant que $n \geq 2$ est une puissance de 2. On découpe $M = \begin{pmatrix} A & B \\ C & D \end{pmatrix}$ avec A, D de taille $\frac{n}{2}$.

39. Évaluer les complexités des sommes et produits. En déduire $C(n) = 4C(\frac{n}{2}) + O(n^3)$, puis la complexité de l'algorithme.

40. Comment gérer une taille n quelconque ? Quelle complexité peut-on obtenir pour M^* ?

41. Écrire `etoile : mat -> mat` qui renvoie l'étoile d'une matrice avec l'algorithme récursif le plus adéquat.

II.C – Algorithme de Conway

Soit $A = (Q, I, F, T)$ avec $Q = \llbracket 0, n-1 \rrbracket$. Sa matrice de transition est la matrice d'expressions régulières M_A dont le coefficient $[M_A]_{i,j}$ est l'union, notée $\mid$, des lettres $c \in \Sigma$ telles que $(i, c, j) \in T$. L'union vide vaut $\emptyset$. On admet que $\mathcal{L}([M_A^*]_{i,j}) = L_{i,j}$, langage des chemins défini à la question 9.

42. Montrer que $L_A = \mathcal{L}([XM_A^*Y]_{0,0})$, où X est une matrice ligne $(x_0, \dots, x_{n-1})$ et Y une matrice colonne de coefficients $y_0, \dots, y_{n-1}$, avec $x_i, y_j \in \{\emptyset, \varepsilon\}$. Préciser les coefficients en fonction de l'automate.

43. Écrire `langage : automate -> exprat` renvoyant une expression régulière du langage de l'automate. Quelle est sa complexité ?

III – Automate des dérivées d'Antimirov

On propose une méthode construisant, à partir d'une expression régulière, un automate non déterministe ayant peu d'états. Pour deux ensembles d'expressions S, S' , on pose

$$S \cdot S' = \{E \cdot E' \mid (E, E') \in S \times S'\}.$$

En particulier, $\emptyset \cdot S = \emptyset$ et $\{\varepsilon\} \cdot S = S$. On utilise les conventions syntaxiques d'associativité de la concaténation et de neutralité de ε . Soient E une expression régulière sur Σ et $a \in \Sigma$. La dérivée partielle $\partial_{a(E)}$ est l'ensemble d'expressions défini inductivement par

$$\begin{aligned} \partial_{a(\emptyset)} &= \emptyset, \quad \partial_{a(\varepsilon)} = \emptyset, \quad \partial_{a(b)} = \begin{cases} \{\varepsilon\} & \text{si } a = b \\ \emptyset & \text{sinon} \end{cases}, \\ \partial_{a(E \mid F)} &= \partial_{a(E)} \cup \partial_{a(F)}, \quad \partial_{a(E^*)} = \partial_{a(E)} \cdot \{E^*\}, \\ \partial_{a(EF)} &= \begin{cases} \partial_{a(E)} \cdot \{F\} & \text{si } \varepsilon \notin \mathcal{L}(E) \\ \partial_{a(E)} \cdot \{F\} \cup \partial_{a(F)} & \text{sinon.} \end{cases} \end{aligned}$$

La dérivée partielle prend donc une expression en argument et renvoie un ensemble d'expressions.

Par exemple, pour $E = a^*(a \mid b)$, puisque $\varepsilon \in \mathcal{L}(a^*)$,

$$\begin{aligned} \partial_{a(E)} &= \partial_{a(a^*)} \cdot \{a \mid b\} \cup \partial_{a(a \mid b)} = \{a^*(a \mid b), \varepsilon\}, \\ \partial_{b(E)} &= \partial_{b(a^*)} \cdot \{a \mid b\} \cup \partial_{b(a \mid b)} = \{\varepsilon\}. \end{aligned}$$

44. Pour $E = (ab \mid b)^*ba$, calculer $\partial_{a(E)}$ et $\partial_{b(E)}$.

On étend les dérivées aux mots et aux ensembles d'expressions : pour $a \in \Sigma$, $w \in \Sigma^*$ et un ensemble d'expressions S ,

$$\partial_{\varepsilon(E)} = \{E\}, \quad \partial_{wa}(E) = \partial_{a(\partial_w(E))}, \quad \partial_{w(S)} = \bigcup_{E \in S} \partial_w(E).$$

L'automate d'Antimirov de E est $A = (Q, I, F, T)$, défini par

$$\begin{aligned} Q &= \{E_1 \mid \exists w \in \Sigma^*, E_1 \in \partial_w(E)\}, \quad I = \{E\}, \\ F &= \{E_1 \in Q \mid \varepsilon \in \mathcal{L}(E_1)\}, \\ T &= \{(E_1, c, E_2) \in Q \times \Sigma \times Q \mid E_2 \in \partial_{c(E_1)}\}. \end{aligned}$$

Pour tout mot w et tout langage $L \subseteq \Sigma^*$, on rappelle $w^{-1}L = \{u \in \Sigma^* \mid wu \in L\}$.

45. Dessiner l'automate obtenu à partir de $E = (ab \mid b)^*ba$. Indiquer précisément son ensemble d'états Q .

46. Montrer que pour tous mots u, v et tout langage L , $v^{-1}(u^{-1}L) = (uv)^{-1}L$.

Pour un ensemble d'expressions S , on note $\mathcal{L}(S)$ la réunion de leurs langages. On admet que pour toute expression E et toute lettre x ,

$$\mathcal{L}(\partial_{x(E)}) = x^{-1}\mathcal{L}(E).$$

47. Pour tout ensemble S d'expressions sur Σ et tout $w \in \Sigma^*$, montrer $\mathcal{L}(\partial_{w(S)}) = w^{-1}\mathcal{L}(S)$.

48. Montrer que pour tout $w \in \Sigma^*$, $\partial_w(E)$ est l'ensemble des états accessibles depuis E en lisant w .

49. En déduire que l'automate d'Antimirov reconnaît le langage de E .

Pour $w \in \Sigma^* \setminus \{\varepsilon\}$ et toutes expressions E, F , on vérifie les relations

$$\partial_{w(E \mid F)} = \partial_{w(E)} \cup \partial_{w(F)} \quad (\text{III.1}),$$

$$\partial_{w(EF)} \subseteq \partial_{w(E)} \cdot \{F\} \cup \bigcup_{v \in S^+(w)} \partial_v(F) \quad (\text{III.2}),$$

$$\partial_{w(E^*)} \subseteq \bigcup_{v \in S^+(w)} \partial_v(E) \cdot \{E^*\} \quad (\text{III.3}),$$

où $S^+(w)$ est l'ensemble des suffixes non vides de w . On pose

$$Q(E) = \cup_{w \in \Sigma^* \setminus \{\varepsilon\}} \partial_{w(E)}.$$

50. Montrer que $|Q(E)|$ est majoré par le nombre de lettres présentes dans l'écriture syntaxique de E , noté $\|E\|$. Qu'en déduit-on sur l'automate d'Antimirov ?